Final Report: MLSE Project
Event-Type Player Similarity in the PlayerBERT Pipeline

Ziheng Lin

March 25, 2026

1 1. Intro

This report investigates the following question: *for a given player, who is most similar under a specific event type, such as `Pass` or `Carry`?* The question arises from a limitation of global player similarity, in which a single embedding per player conflates many distinct behaviors and therefore obscures action-specific structure. To address this issue, I develop an end-to-end pipeline built on StatsBomb open data and 360 context, including data preprocessing, event-level representation learning, sequence-level modeling, and a downstream retrieval stage; the primary emphasis of the report is the Event-Type Player Similarity component because it provides the most direct answer to the research question.

2 2. Data & Data Processing

2.1 Data used

The project uses StatsBomb open data and relies on three aligned sources from the repository:

- event JSON files from `open-data/data/events`,
- 360 freeze-frame JSON files from `open-data/data/three-sixty`,
- lineup JSON files from `open-data/data/lineups`.

These sources are merged to produce the processed file `events360_v4.jsonl`, which serves as the principal input for EventEncoder training, PlayerBERT training, and the downstream event-type similarity cache.

Table 1: Data sources used in the pipeline.

Source	Role in the pipeline
StatsBomb event files	Core event metadata such as type, player, team, timestamp, pass fields, shot fields, and raw locations.
StatsBomb 360 frames	Local spatial context around the event through `freeze_frame` and visible area.
Lineup files	Positional metadata used to replace `pass.recipient` with `recipient_position`.

2.2 Processing steps

The preprocessing logic is implemented in `scripts/preprocess_360_events.py`. The pipeline proceeds in the following stages:

1. Match each 360 frame to its event using the shared event identifier, `event_uuid`.
2. Copy event attributes and append 360 context, especially `freeze_frame` and `visible_area`.
3. Load lineup data and replace `pass.recipient` with `pass.recipient_position` so that tactical role is preserved while direct recipient identity is removed.
4. Normalize boolean fields so missing values become `False`.
5. Normalize categorical fields so missing values become `Unknown`.
6. Bucketize continuous variables into coarse categories, including top-level location, pass end location, pass length, pass angle, shot end location, and shot xG.
7. Keep actor location and freeze-frame lists because they are consumed directly by the EventEncoder.
8. Write the final enriched dataset as one JSON object per line to `events360_v4.jsonl`.

These design choices are intended to improve both statistical stability and representational consistency. Bucketization reduces the cardinality of tabular features, normalization ensures that missingness is encoded consistently across events, and the use of recipient position instead of recipient identity helps retain tactical information while reducing direct player-identity leakage.

3 3. Methods

3.1 Overview

The full pipeline contains three connected modeling components: an EventEncoder, a sequence-level PlayerBERT model, and an event-type-conditioned retrieval layer. Their respective roles are summarized in Table 2.

Table 2: Methods used in the project and their motivation.

Model / component	Motivation and utility
EventEncoder	Learn a contextualized embedding for a single event by combining event attributes with 360 spatial context. This is the core representation used throughout the project.
PlayerBERT	Model ordered per-player event sequences within matches so that temporal structure is captured, rather than treating events independently.
Event-Type Player Similarity	Reuse the pretrained EventEncoder to build player profiles conditioned on one event type, so the system can answer focused questions such as who passes like a given player.

3.2 EventEncoder

The EventEncoder is trained using masked attribute modeling and combines three complementary components:

- a Transformer encoder over discrete event features,
- an MLP-based set encoder over 360 freeze-frame players,
- a gated fusion layer that merges event information and frame information.

The event branch captures *what* happened through structured event attributes, while the frame branch captures *where* the action took place relative to surrounding players. A gating mechanism fuses these two views into a single event representation in $\mathbb{R}^{128}$.

3.3 PlayerBERT

PlayerBERT operates at the sequence level. It groups events by player and match, orders them by timestamp, and applies masked event modeling over event embeddings. In the current pipeline, the EventEncoder is kept frozen and used as the embedding initializer. This stage is intended to capture within-match temporal structure and thereby support more global player representations than event-level modeling alone.

3.4 Event-Type Player Similarity

The main method studied in this report is Event-Type Player Similarity. For each event with player ID p and event type t, the pretrained EventEncoder produces an embedding $\mathbf{e}_i$. These embeddings are then aggregated by player and event type to form a profile vector:

$$\mathbf{v}_{p,t} = \frac{\sum_i \mathbf{e}_i}{n_{p,t}},$$

where $n_{p,t}$ is the number of stored events for player p under event type t.

This formulation is useful because it addresses behavior-specific questions rather than broad whole-player comparisons. It is also computationally efficient, since the implementation stores only one running sum and one count for each distinct (p, t) pair instead of materializing every event embedding.

At query time, the system compares a query profile against all valid candidate profiles for the same event type using cosine similarity. Both the query player and the candidate player must satisfy a minimum support threshold, `min_count`, so that retrieval is grounded in a sufficient number of observed events.

4 4. Results

4.1 Built cache summary

The current cache was constructed from `events360_v4.jsonl` using the pretrained EventEncoder checkpoint. Table 3 summarizes the resulting artifact.

Table 3: Summary statistics of the built event-type profile cache.

Metric	Value
JSONL rows read	1,027,908
Events encoded	1,027,857
Skipped (no `player.id`)	51
Distinct players	2,409
Distinct event types	22
Stored (p, t) profile cells	31,039

These statistics indicate that the retrieval layer scales to more than one million event rows while still producing a compact cache of slightly more than thirty-one thousand player-event-type profiles.

4.2 Event-type distribution

Figure 1 shows the total count of events per event type in the profile cache.

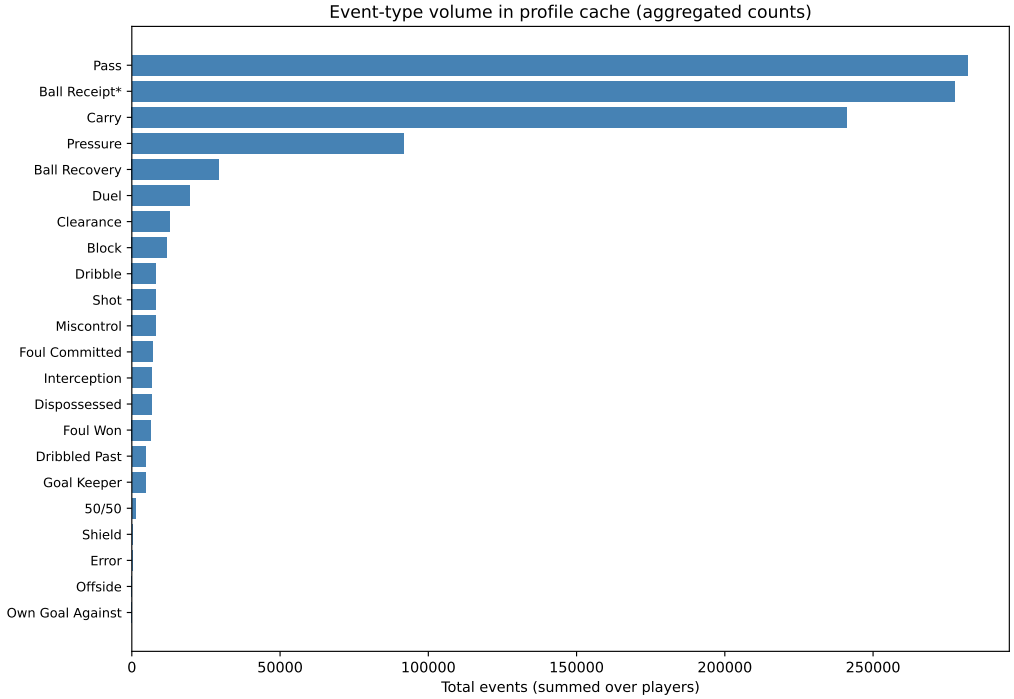

Figure 1: Total number of stored events by event type in the event-type profile cache. The distribution is head-heavy, with Pass, Ball Receipt*, and Carry dominating the cache.

This figure is directly relevant to the research question because event types with larger support, especially `Pass` and `Carry`, permit more stable event-type-conditioned comparisons across a large number of players.

4.3 Support distribution across profiles

Figure 2 shows the support distribution over all stored (p, t) profiles.

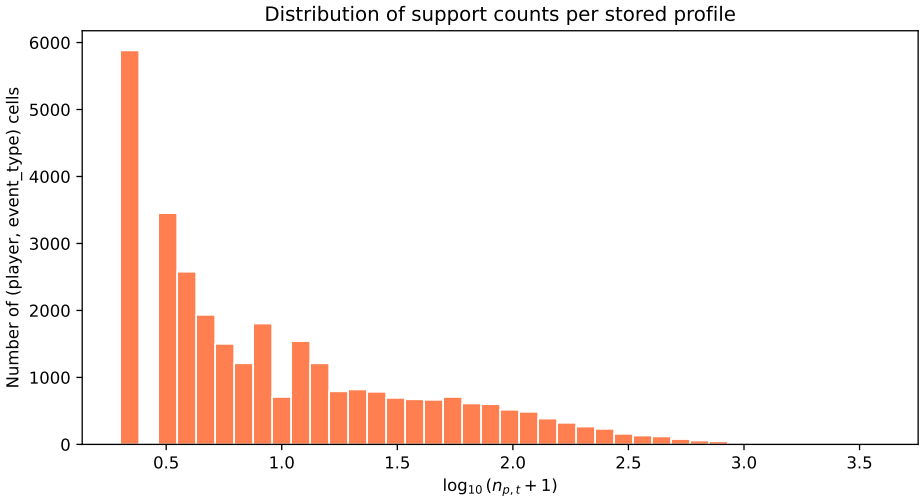

Figure 2: Distribution of support counts across stored player-event-type profiles, plotted on a log scale. This motivates the use of a minimum count threshold during querying.

The figure shows that many player-event-type cells have moderate support, while a smaller subset has very high support. This pattern justifies the use of `min_count` in order to avoid unstable retrieval for rare event types.

4.4 Example retrieval result

Table 4 shows one `Pass` query example from the slides, using `top-k=5` and `min-count=20`.

Table 4: Example nearest neighbors for a pass-profile query.

Player	Similarity	Count
Alexandra Morgan Carrasco	0.9850	75
Borja Iglesias Quintas	0.9844	74
Harry Kane	0.9913	21
Aitana Bonmati Conca	0.9899	37
Alphonso Davies	0.9899	22

This result is important for interpretation. The nearest neighbors can span leagues and genders because the model clusters *how* a pass appears in context rather than *who* the player is demographically. This is precisely why event-type-conditioned similarity is more informative than a single global embedding when the research question is action-specific.

5 5. What comes next?

Several extensions would strengthen the project further. First, profile aggregation could be made more expressive by replacing simple means with trimmed means or weighted means. Second, the same framework could be conditioned on finer facets such as `pass.height.name` or body part. Third, league-aware normalization could improve comparability across competitions. Finally, the event-

type retrieval layer could be integrated more directly with PlayerBERT outputs so that action-level specificity and temporal structure are modeled jointly.

A 6. Appendix: technical aspects of the methods

A.1 Assumptions behind data processing

The preprocessing pipeline relies on several assumptions:

- event IDs and 360 frame IDs are aligned correctly through `event_uuid`,
- replacing pass recipients with recipient positions preserves tactical meaning while reducing identity leakage,
- bucketizing continuous values is acceptable because coarse tactical categories are more important than exact raw values for this task,
- missing booleans and categoricals can be normalized safely into default tokens.

A.2 Assumptions behind the models

The EventEncoder assumes that a single event can be represented adequately through a combination of tabular event attributes and local 360 context. The frame encoder further assumes that mean-pooling over visible players is sufficient to summarize the local spatial neighborhood.

PlayerBERT assumes that ordered event sequences within a match contain meaningful temporal structure and that a Transformer encoder can learn this structure from event embeddings.

Event-Type Player Similarity assumes that averaging event embeddings within one event type is sufficient to produce a meaningful marginal profile for a player under that action category.

A.3 Alignment between modeling and processed data

The models depend directly on the processed data schema. EventEncoder training uses discrete feature IDs derived from flattened and normalized event fields, while the frame encoder consumes the actor location and the freeze-frame player list. Consequently, the `feature_vocab` saved in `event_encoder_mam.pt` must match the exact processed feature space observed at inference time.

A.4 Model and data limitations

The principal limitations are:

- mean pooling may mix multiple sub-styles within the same event type,
- rare event types produce noisier retrieval even when they meet a low support threshold,
- the current retrieval method does not model temporal order directly,
- the quality of the learned representation depends on consistency between the preprocessing pipeline and the saved checkpoint.

Despite these limitations, the event-type retrieval layer remains a useful and interpretable downstream application of the learned event representations.